

Vehicle Management Backend

Complete Documentation

A Beginner-Friendly Guide for Node.js + Express + MongoDB Backend Development

Document Created: 8/31/2026

1. PROJECT INTRODUCTION

What This Backend Does

This backend is a Smart Vehicle Service Platform that helps customers book vehicle service appointments at garages. Customers can:

- Register and login with an account
- Add and manage their vehicles
- Discover nearby garages and their services
- Book appointments at garages
- Communicate with mechanics via chat
- Write reviews for completed services
- View invoices and payment status

Garage owners can:

- Create and manage their garages
- Add mechanics to their team
- Manage service offerings and pricing
- Accept and update bookings

What is Vehicle Management?

Vehicle Management means organizing and handling vehicle-related services. In this system, it includes:

- Storing customer vehicle information (brand, model, number)
- Managing which user owns which vehicle
- Booking service appointments for vehicles
- Tracking service history through invoices

Technologies Used

1. Node.js - JavaScript runtime that allows JavaScript to run on servers
2. Express - Web framework that makes creating APIs easy
3. MongoDB - NoSQL database that stores data as JSON documents
4. Mongoose - Library that helps us work with MongoDB in Node.js
5. bcrypt - Security library for encrypting passwords
6. JWT (JSON Web Token) - Used for secure authentication and session management

Why Each Technology?

- Node.js: Allows JavaScript on servers. Easy to learn and very fast.
- Express: Simplifies handling HTTP requests/responses. Makes routing easy.
- MongoDB: Flexible database that stores data in JSON format (documents). No need to define fixed table structures.
- Mongoose: Makes it easier to define data structure (schemas) and validate data.
- bcrypt: Hashes passwords so they cannot be read even if database is hacked.
- JWT: Creates tokens that prove a user is logged in without storing session on server.

2. PROJECT STRUCTURE

The actual project folder structure:

```
my-backend/
%
% % % server.js           (Main server file - starts everything)
% % % package.json       (Project dependencies)
% % % dropDatabase.js     (Script to drop MongoDB database)
%
% % % config/
%   % % % db.js           (MongoDB connection setup)
%
% % % models/            (Data structure definitions)
%   % % % User.js         (User schema - customers, mechanics, owners)
%   % % % Vehicle.js      (Vehicle schema - cars owned by customers)
%   % % % Booking.js      (Booking schema - appointments)
%   % % % Garage.js       (Garage schema - service centers)
%   % % % Invoice.js       (Invoice schema - billing)
%   % % % Review.js       (Review schema - customer feedback)
%   % % % Message.js      (Message schema - chat between users)
%   % % % Notification.js (Notification schema - user alerts)
%
% % % controllers/       (Business logic - handles requests)
%   % % % userController.js (User login, register, CRUD)
%   % % % vehicleController.js (Vehicle CRUD operations)
%   % % % bookingController.js (Booking CRUD operations)
%   % % % garageController.js (Garage CRUD operations)
%   % % % platformController.js (Reviews, messages, notifications)
%
% % % routes/            (API endpoints - what URLs exist)
%   % % % userRoutes.js   (Register, login, get users)
%   % % % vehicleRoutes.js (Vehicle endpoints)
%   % % % bookingRoutes.js (Booking endpoints)
%   % % % garageRoutes.js (Garage endpoints)
%   % % % platformRoutes.js (Review, chat, notification endpoints)
%
% % % middleware/
%   % % % auth.js         (Authentication & authorization checks)
```

What Each Folder Does:

models/

Contains Mongoose schemas - these define what data each collection stores and what data types are required. Think of models as the blueprint for data.

controllers/

Contains the business logic - the actual code that handles API requests. Controllers talk to models to read/write data from MongoDB.

routes/

Maps URLs to controllers. When someone makes a request to /users/register, the routes tell Express which controller function to run.

config/

Contains configuration files, like database connection settings.

middleware/

Contains middleware - code that runs before reaching the controller. auth.js checks if user is logged in.

3. SERVER.JS EXPLANATION

What does server.js do?

server.js is the entry point - it starts the entire backend. Here is the flow:

1. Load environment variables from .env file
2. Import Express framework
3. Import database connection function
4. Import all route files
5. Create Express app
6. Add middleware (express.json())
7. Register all routes
8. Connect to MongoDB
9. Start listening on a port (usually 3000)

The Backend Flow:

Postman (or browser)

!“

HTTP Request to /users/register

!“

Express Routes

!“

userController.registerUser()

!“

User.js (Mongoose Model)

!“

MongoDB Database

!“

HTTP Response (JSON)

What happens in server.js:

- app.use(express.json()) - tells Express to accept JSON data from requests
- app.use("/users", userRoutes) - all URLs starting with /users use userRoutes
- connectDB() - connects to MongoDB server
- app.listen(3000, ...) - server listens on port 3000 for incoming requests

4. DATABASE EXPLANATION

What is MongoDB?

MongoDB is a database (like Excel for programmers). It stores data in collections (like sheets). Each row of data is called a document (like JSON).

What is Mongoose?

Mongoose is a library that helps us work with MongoDB. It lets us:

- Define schemas (structure of data)
- Validate data (check if required fields exist)
- Query data (find, update, delete)
- Create relationships between data

What is a Schema?

A schema is the blueprint. It says: "A User has a name (String), email (String), password (String)"

What is a Collection?

A collection is like a table. It contains many documents. MongoDB database "vehicleManagement" has 8 collections.

Collections in vehicleManagement database:

1. users - stores all user accounts
2. vehicles - stores customer vehicles
3. bookings - stores service appointments
4. garages - stores garage/service center information
5. invoices - stores billing information
6. reviews - stores customer reviews
7. messages - stores chat messages
8. notifications - stores user notifications

How Data Reaches MongoDB:

Step 1: User sends request via Postman

POST http://localhost:3000/users/register

Body: { "name": "John", "email": "john@test.com", "password": "12345678" }

Step 2: Express receives request, routes to userController

Step 3: userController.registerUser() function runs

```
const newUser = new User({ name, email, password: hashedPassword });  
await newUser.save();
```

Step 4: Mongoose connects to MongoDB

Step 5: MongoDB stores document in users collection

Step 6: Response sent back to Postman with user ID and JWT token

Database Connection:

Connection string: mongodb://127.0.0.1:27017/vehicleManagement

- 127.0.0.1 = localhost (your computer)
- 27017 = default MongoDB port
- vehicleManagement = database name

5. DATA MODELS (SCHEMAS)

Model 1: User

Collection: users
Purpose: Stores all user accounts (customers, mechanics, garage owners)

Field	Type	Required	Purpose
name	String	Yes	User full name
email	String	Yes (Unique)	User email - must be unique
mobile	String	Yes	User phone number
role	String	No	Role: customer, garage_owner, mechanic, admin
password	String	Yes	Encrypted password using bcrypt
timestamps	Date	Auto	createdAt and updatedAt

Model 2: Vehicle

Collection: vehicles
Purpose: Stores vehicles owned by customers

Field	Type	Required	Purpose
owner	ObjectId	Yes	Reference to user
vehicleNumber	String	Yes	Licence plate number
brand	String	Yes	Car brand
model	String	Yes	Car model
fuelType	String	Yes	Petrol, Diesel, Electric
manufacturingYear	Number	Yes	Year of manufacture

Rel

Model 3: Booking

Collection: bookings

Purpose: Stores service appointment bookings

Field	Type	Purpose
customer	ObjectId	Reference to User (who booked)
vehicle	ObjectId	Reference to Vehicle (which car)
garage	ObjectId	Reference to Garage (where booked)
mechanic	ObjectId	Reference to User (assigned mechanic)
service	Object	{ name: String, price: Number }
appointmentAt	Date	When appointment is scheduled
status	String	pending, confirmed, in-progress, completed
notes	String	Additional notes about booking
completedAt	Date	When service was completed

Relationships:

- Booking !' Customer (User) !' Can have many bookings
- Booking !' Vehicle !' Each booking is for one vehicle
- Booking !' Garage !' Each booking is at one garage
- Booking !' Mechanic (User) !' Assigned mechanic handles booking

Model 4: Garage

Collection: garages

Purpose: Stores garage/service center information

Field	Type	Purpose
owner	ObjectId	Reference to User (garage owner)
name	String	Garage name (e.g., "ABC Car Service")
address	String	Full address of garage
phone	String	Garage contact number
description	String	Details about garage services
services	Array	List of services with { name, description, price }
mechanics	Array	List of mechanic User IDs working in garage
rating	Number	Average rating (0-5)
verified	Boolean	Is garage verified by platform

Model 5: Invoice

Collection: invoices

Purpose: Stores billing information for completed bookings

Field	Type	Purpose
booking	ObjectId	Reference to Booking
customer	ObjectId	Reference to User (customer)
garage	ObjectId	Reference to Garage
mechanic	ObjectId	Reference to User (mechanic)
vehicle	ObjectId	Reference to Vehicle
serviceCharges	Number	Cost of service
sparePartsCost	Number	Cost of spare parts used
tax	Number	Tax amount
total	Number	serviceCharges + sparePartsCost + tax
paymentStatus	String	pending or paid

Model 6: Review

Collection: reviews

Purpose: Customer reviews for completed services

Field	Type	Purpose
booking	ObjectId	Reference to Booking being reviewed
customer	ObjectId	Reference to User (who wrote review)
garage	ObjectId	Reference to Garage (being reviewed)
rating	Number	Rating from 1 to 5 stars
comment	String	Written review text

Model 7: Message

Collection: messages

Purpose: Chat messages between customer and mechanic for a booking

Field	Type	Purpose
booking	ObjectId	Reference to Booking (which appointment)
sender	ObjectId	Reference to User (who sent message)
recipient	ObjectId	Reference to User (who receives message)
text	String	Message text content
imageUrl	String	URL to image (optional)

Model 8: Notification

Collection: notifications

Purpose: Alerts sent to users (booking updates, reviews, etc)

Field	Type
recipient	ObjectId
type	String
message	String
read	Boolean
booking	ObjectId

6. CONTROLLERS & BUSINESS LOGIC

Controllers contain the actual logic that handles API requests. Each controller has functions for CRUD operations:

- C = CREATE - Add new data
- R = READ - Get data
- U = UPDATE - Modify data
- D = DELETE - Remove data

UserController

File: controllers/userController.js

Functions:

- registerUser() - Creates new user account with hashed password
- loginUser() - Verifies email/password and returns JWT token
- getUsers() - Gets all users (admin only)
- updateUser() - Updates user profile
- deleteUser() - Deletes user account (admin only)

VehicleController

File: controllers/vehicleController.js

Functions:

- listVehicles() - Get all vehicles of logged-in customer
- addVehicle() - Customer adds a new vehicle
- updateVehicle() - Customer updates vehicle details
- deleteVehicle() - Customer deletes a vehicle

Note: Only customers can manage vehicles. Each vehicle is tied to vehicle owner.

BookingController

File: controllers/bookingController.js

Functions:

- listBookings() - Get bookings (filters by user role)
 - Customers see their own bookings
 - Garage owners see bookings for their garages
 - Mechanics see bookings assigned to them
- createBooking() - Customer creates appointment booking
- updateBooking() - Update booking status or appointment time

GarageController

File: controllers/garageController.js

Functions:

- createGarage() - Garage owner creates a new garage
- listOwnedGarages() - Get garages owned by logged-in user
- updateGarage() - Garage owner updates garage details
- addMechanic() - Garage owner adds a mechanic to their garage

Note: Only garage owners can create/manage garages.

PlatformController

File: controllers/platformController.js

Functions:

- discoverGarages() - Public: Search and discover garages
- getGarage() - Public: Get details of one garage
- createReview() - Logged-in customer reviews completed booking
- listReviews() - Get reviews for a garage
- updateReview() - Customer updates their review
- deleteReview() - Customer deletes their review
- listNotifications() - Get notifications for logged-in user
- markNotificationRead() - Mark notification as read
- listMessages() - Get chat messages for a booking
- sendMessage() - Send chat message for a booking
- listInvoices() - Customer gets their invoices
- getInvoice() - Download invoice details

7. COMPLETE API REFERENCE

All endpoints in the backend. Authorization column shows who can access.

7.1 USER ENDPOINTS

Method	URL	Purpose	Auth	Body
POST	/users/register	Create new user	No	name, email, mobile,
POST	/users/login	Login user	No	email, password
GET	/users	Get all users	Admin	none
PUT	/users/:id	Update user	Logged in	name, email, mobile
DELETE	/users/:id	Delete user	Admin	none

7.2 VEHICLE ENDPOINTS

All vehicle endpoints require login with customer role

Method	URL
GET	/vehicles
POST	/vehicles
PUT	/vehicles/:id
DELETE	/vehicles/:id

7.5 PLATFORM ENDPOINTS (PUBLIC & PRIVATE)

Method	URL	Purpose	Auth	Body
GET	/api/garages	Search garages	No	query: service or se
GET	/api/garages/:id	Get garage details	No	none
GET	/api/garages/:id/reviews	Get garage reviews	No	none
POST	/api/reviews	Write review	Customer	booking, rating, com
PUT	/api/reviews/:id	Update review	Customer	rating, comment
DELETE	/api/reviews/:id	Delete review	Customer	none
GET	/api/notifications	Get notifications	Login	none
PATCH	/api/notifications/:id/read	Mark as read	Login	none
GET	/api/chat/:bookingId	Get messages	Login	none
POST	/api/chat/:bookingId	Send message	Login	text, imageUrl
GET	/api/invoices	Get invoices	Customer	none
GET	/api/invoices/:id/download	Download invoice	Customer	none

8. AUTHENTICATION & JWT EXPLANATION

What is Authentication?

Authentication = Proving you are who you say you are. Usually with username/password.

What is Authorization?

Authorization = After proving who you are, what are you allowed to do?

Example: A customer can only view their own vehicles, not other people's vehicles.

Registration Flow:

1. User sends: POST /users/register { name, email, password }
2. Server checks if email already exists
3. Server hashes password using bcrypt (converts "12345678" to "\$2b\$10\$...")
4. Server saves user to database
5. Server creates JWT token
6. Server returns token to client

What is JWT (JSON Web Token)?

A JWT is a secure token that proves a user is logged in. It looks like:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI123... .

It contains:

- Header - encryption method
- Payload - user info (userId, role)
- Signature - secret key that verifies token is real

Login Flow:

1. User sends: POST /users/login { email, password }
2. Server finds user by email in database
3. Server uses bcrypt.compare() to check if password matches
4. If password matches, server creates JWT token
5. Server returns JWT token
6. Client stores JWT (in localStorage, cookies, etc)

Using the Token in Requests:

After login, client sends JWT in every request:

- In Header: Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
- Server verifies token using middleware (auth.js)
- If token is valid, request proceeds
- If token is invalid or expired, request is rejected (401 error)

Why JWT instead of Sessions?

- JWT is stateless - no need to store sessions on server
- Scalable - works with multiple servers
- Secure - token is signed and verified
- Mobile friendly - works on apps and web

9. AUTHORIZATION & ROLES

User Roles in This System:

Role	Can Do	Cannot Do
customer	Register, login, add vehicles, book appointments, write reviews, view	Create garage, assign mechanics, manage other
garage_owner	Create garage, add mechanics, manage services, update bookings	Book services, review other garages
mechanic	Accept bookings, update booking status, send messages	Create garage, manage vehicles
admin	Everything - view all users, delete accounts, manage entire system	Nothing - full access

How Authorization Works:

- Middleware checks user role from JWT token
- allowRoles("customer") - only allows customers
- allowRoles("garage_owner") - only allows garage owners
- If user doesn't have required role, gets 403 error: "You are not authorized"

Authorization Examples:

- Only customers can create bookings
- Only garage owners can create garages
- Only customer who owns a vehicle can update it
- Only customer who booked can leave review for that booking
- Only admin can see all users

10. COMPLETE POSTMAN TESTING GUIDE

This step-by-step guide will help you test every API endpoint.

Prerequisites: MongoDB running, backend running on <http://localhost:3000>

Step 1: Start MongoDB

Open MongoDB Compass or start MongoDB service

Connection: `mongodb://127.0.0.1:27017`

Step 2: Start Backend

```
cd d:\CODEEING\AT\VehicleServicesManagementSystem\my-backend
node server.js
```

Should see: "Server running on port 3000"

Step 3: Open Postman

Create new workspace or open existing one

Set Base URL to: <http://localhost:3000>

Step 4: TEST USER REGISTRATION

Create new request:

Method: POST

URL: <http://localhost:3000/users/register>

Headers:

Content-Type: application/json

Body (JSON):

```
{
  "name": "Test Customer",
  "email": "customer@test.com",
  "mobile": "9876543210",
  "password": "password123",
  "role": "customer"
}
```

Expected Response: 201 Created with user data and JWT token

Copy the token from response - you'll need it for next requests

Also register these users:

Garage Owner:

email: owner@test.com, role: garage_owner

Mechanic:

email: mechanic@test.com, role: mechanic

Step 5: TEST USER LOGIN

Method: POST

URL: http://localhost:3000/users/login

Headers: Content-Type: application/json

Body:

```
{
  "email": "customer@test.com",
  "password": "password123"
}
```

Expected: 200 OK with user data and token

This token is used for all authenticated requests

Step 6: TEST ADD VEHICLE

Method: POST

URL: http://localhost:3000/vehicles

Headers:

Content-Type: application/json

Authorization: Bearer <paste_customer_token>

Body:

```
{
  "vehicleNumber": "MH12TEST01",
  "brand": "Toyota",
  "model": "Innova",
  "fuelType": "Diesel",
  "manufacturingYear": 2020
}
```

Expected: 201 Created with vehicle details including vehicleId

Copy vehicleId for booking creation

Step 7: TEST GET VEHICLES

Method: GET

URL: http://localhost:3000/vehicles

Headers: Authorization: Bearer <customer_token>

Expected: 200 OK with list of vehicles

Step 8: CREATE GARAGE (as garage_owner)

Method: POST

URL: http://localhost:3000/garages

Headers:

Content-Type: application/json

Authorization: Bearer <paste_garage_owner_token>

Body:

```
{
  "name": "ABC Car Service",
  "address": "123 Main Street, City",
  "phone": "9999999999",
  "description": "Professional car service center",
  "services": [
    {
      "name": "Oil Change",
      "description": "Complete oil change service",
      "price": 500,
      "durationMinutes": 30
    },
    {
      "name": "General Service",
      "description": "Complete vehicle checkup",
      "price": 2000,
      "durationMinutes": 120
    }
  ]
}
```

Expected: 201 Created with garage details and garageld

Copy garageld for booking creation

Step 9: ADD MECHANIC TO GARAGE (as garage_owner)

Method: POST

URL: `http://localhost:3000/garages/<garageId>/mechanics`

Headers: Authorization: Bearer <garage_owner_token>

Body:

```
{
  "mechanicId": "<mechanic_user_id>"
}
```

Note: Get mechanic user ID from step where mechanic registered

Expected: 200 OK with updated garage including mechanics list

Step 10: DISCOVER GARAGES (public)

Method: GET

URL: `http://localhost:3000/api/garages`

Headers: No authorization needed

Optional query:

?service=Oil Change - filter by service

?search=ABC - search by garage name/address

Expected: 200 OK with list of all garages

Step 11: CREATE BOOKING (as customer)

Method: POST

URL: `http://localhost:3000/bookings`

Headers: Authorization: Bearer <customer_token>

Body:

```
{
  "vehicle": "<vehicleId>",
  "garage": "<garageId>",
  "mechanic": "<mechanicId>",
  "service": "Oil Change",
  "appointmentAt": "2024-09-15T10:00:00Z",
  "notes": "Please check brakes too"
}
```

Note: appointmentAt must be in future

Expected: 201 Created with booking details and bookingId

Copy bookingId for messages and invoice

Step 12: GET BOOKINGS

Method: GET

URL: `http://localhost:3000/bookings`

Headers: Authorization: Bearer <token>

Results filter by role:

- Customer: Sees only their bookings
- Garage Owner: Sees bookings for their garages
- Mechanic: Sees bookings assigned to them

Expected: 200 OK with filtered list

Step 13: UPDATE BOOKING STATUS (as mechanic/garage owner)

Method: PATCH

URL: `http://localhost:3000/bookings/<bookingId>`

Headers: Authorization: Bearer <mechanic_or_owner_token>

Body:

```
{
  "status": "in-progress"
}
```

Valid statuses: pending, confirmed, in-progress, completed

Expected: 200 OK with updated booking

Step 14: SEND CHAT MESSAGE

Method: POST

URL: `http://localhost:3000/api/chat/<bookingId>`

Headers: Authorization: Bearer <customer_or_mechanic_token>

Body:

```
{
  "text": "When will my car be ready?",
  "imageUrl": "https://example.com/image.jpg" [optional]
}
```

Expected: 201 Created with message details

Step 15: GET MESSAGES FOR BOOKING

Method: GET

URL: `http://localhost:3000/api/chat/<bookingId>`

Headers: Authorization: Bearer <token>

Expected: 200 OK with all messages for that booking

Step 16: WRITE REVIEW (after booking completed)

Method: POST

URL: `http://localhost:3000/api/reviews`

Headers: Authorization: Bearer <customer_token>

Body:

```
{
  "booking": "<bookingId>",
  "rating": 5,
  "comment": "Great service! Very professional mechanics"
}
```

Note: Only works if booking status is "completed"

Expected: 201 Created with review details

Step 17: GET REVIEWS FOR GARAGE

Method: GET

URL: `http://localhost:3000/api/garages/<garageId>/reviews`

Headers: No authorization needed

Expected: 200 OK with all reviews for that garage

Step 18: ERROR TESTING

Test 1: Missing Required Field

Try registering without email:

```
POST /users/register
```

```
{ "name": "John", "password": "123456" }
```

Expected: 400 Bad Request - "Email and mobile are required"

Test 2: Duplicate Email

Try registering with same email twice:

First registration succeeds

Second registration:

Expected: 400 Bad Request - "User already exists"

Test 3: Wrong Password

```
POST /users/login
```

```
{ "email": "customer@test.com", "password": "wrongpassword" }
```

Expected: 401 Unauthorized - "Invalid email or password"

Test 4: Missing Authentication Token

Try to add vehicle without Authorization header:

```
POST /vehicles (no token)
```

Expected: 401 Unauthorized - "Authentication required"

Test 5: Invalid Token

```
POST /vehicles
```

```
Authorization: Bearer invalid_token_here
```

Expected: 401 Unauthorized - "Invalid or expired token"

Test 6: Unauthorized Role

Try to create garage as customer:

```
POST /garages (with customer token)
```

Expected: 403 Forbidden - "You are not authorized"

Test 7: Booking Appointment in Past

```
POST /bookings
```

```
{ "appointmentAt": "2020-01-01T10:00:00Z" }
```

Expected: 400 Bad Request - "Appointment must be in the future"

Test 8: Vehicle Not Found

```
PUT /vehicles/invalid123 (with customer token)
```

Expected: 404 Not Found - "Vehicle not found"

11. CONNECTED TEST DATA PLAN

Use this realistic test data for complete testing:

Users to Create:

User Type	Name	Email	Mobile	Password
Customer 1	Rahul Kumar	rahul@test.com	9876543210	Password123
Customer 2	Priya Singh	priya@test.com	9876543211	Password123
Garage Owner	Amit Patel	amit@test.com	9876543212	Password123
Mechanic 1	Raj Verma	raj@test.com	9876543213	Password123
Mechanic 2	Vikas Singh	vikas@test.com	9876543214	Password123
Admin	Admin User	admin@test.com	9876543215	Password123

12. MONGODB COMPASS VERIFICATION

Use MongoDB Compass to visually verify data in the database:

Opening Compass:

1. Open MongoDB Compass
2. Connect to: mongodb://127.0.0.1:27017
3. In left panel, expand "vehicleManagement" database
4. You'll see 8 collections (if data exists)

Verify User Registration:

1. Click on "users" collection
2. See registered users with hashed passwords
3. Verify fields: name, email, mobile, role
4. Note: password is hashed (looks like: \$2b\$10\$...)

Verify Vehicle Creation:

1. Click on "vehicles" collection
2. See all created vehicles
3. Click on one vehicle document
4. Verify: owner (references user ID), vehicleNumber, brand, model
5. Expand "owner" field - should show it links to user ID

Verify Garage:

1. Click on "garages" collection
2. Click on garage document
3. Verify: name, address, services array, mechanics array
4. Expand "services" - see service objects with name, price, durationMinutes
5. Expand "mechanics" array - shows mechanic user IDs

Verify Booking:

1. Click on "bookings" collection
2. Click on booking document
3. Verify: customer ID, vehicle ID, garage ID, mechanic ID
4. See service object: { name, price }
5. Check appointmentAt date and status field

Verify Invoice:

1. Click on "invoices" collection
2. Click on invoice document
3. Verify calculation: total = serviceCharges + sparePartsCost + tax
4. Check paymentStatus field

Verify Review:

1. Click on "reviews" collection
2. Verify: booking ID, customer ID, garage ID, rating, comment
3. Note: createdAt timestamp auto-added

Verify Messages:

1. Click on "messages" collection
2. Verify: booking ID, sender ID, recipient ID, text, createdAt
3. Can see message timeline by sorting by createdAt

Verify Notifications:

1. Click on "notifications" collection
2. Verify: recipient ID, type, message, read status
3. Check if read field changes when marked as read

13. COMPLETE TESTING CHECKLIST

Use this checklist to ensure everything works:

ENVIRONMENT SETUP

- %j MongoDB is installed and running
- %j Node.js and npm installed
- %j Backend code is in d:\CODEEING\AT\VehicleServicesManagementSystem\my-backend
- %j npm install completed (all dependencies installed)
- %j .env file has correct MongoDB URI (if needed)

DATABASE

- %j MongoDB compass connects successfully
- %j vehicleManagement database is created after first request
- %j All 8 collections appear in compass after tests

SERVER

- %j Backend starts with "node server.js"
- %j No connection errors in console
- %j Server shows "Server running on port 3000"
- %j GET http://localhost:3000 returns status "ok"

AUTHENTICATION

- %j User can register with valid data
- %j Registration fails with missing fields
- %j Registration fails with duplicate email
- %j Password is hashed (not plaintext in database)
- %j User can login with correct email/password
- %j Login fails with wrong password
- %j JWT token is returned on successful login
- %j Protected endpoints reject requests without token
- %j Protected endpoints reject requests with invalid token

USER MANAGEMENT

- %j Can register as customer
- %j Can register as garage_owner
- %j Can register as mechanic
- %j Can login and get JWT
- %j Can update own profile
- %j Customer cannot update other user profile
- %j Admin can get all users list
- %j Non-admin cannot get users list

VEHICLES

- %j Customer can add vehicle
- %j Customer can view their vehicles
- %j Customer can update their vehicle
- %j Customer can delete their vehicle
- %j Mechanic cannot add vehicles
- %j Vehicle is linked to correct owner in database
- %j Cannot add vehicle with duplicate number

GARAGES

- %j Garage owner can create garage
- %j Customer cannot create garage
- %j Garage owner can list their garages
- %j Garage owner can update garage
- %j Garage owner can add mechanic to garage
- %j Public can discover all garages
- %j Public can search garages by service name
- %j Public can search garages by location/name

BOOKINGS

- %j Customer can create booking
- %j Booking requires valid vehicle ID
- %j Booking requires valid garage ID
- %j Booking requires future appointment date
- %j Booking fails if appointment is in past
- %j Garage owner can see bookings for their garage
- %j Mechanic can see assigned bookings
- %j Customer can see their bookings
- %j Status can be updated (pending !' confirmed !' in-progress !' completed)

REVIEWS

- %j Customer can write review after booking completed
- %j Cannot review if booking not completed
- %j Review saved with rating (1-5)
- %j Review saved with comment
- %j Public can view garage reviews
- %j Customer can update own review
- %j Customer can delete own review

MESSAGING

- %j Customer can send message to mechanic
- %j Mechanic can send message to customer
- %j Messages are linked to booking
- %j Messages appear with timestamps
- %j Only booking participants can see messages

NOTIFICATIONS

- %j Notifications created for booking updates
- %j User can view their notifications
- %j User can mark notification as read

INVOICES

- %j Invoice created after booking completed
- %j Invoice shows correct amounts
- %j Customer can view their invoices
- %j Invoice shows payment status

ERROR HANDLING

- %j Missing required field returns 400 error
- %j Non-existent ID returns 404 error
- %j Unauthorized access returns 403 error
- %j Invalid token returns 401 error
- %j Server error returns 500 error
- %j All errors return descriptive message

DATA INTEGRITY

- %j MongoDB shows correct data for all users
- %j Relationships between collections work
- %j Timestamps (createdAt, updatedAt) auto-populate
- %j Deleted records removed from database
- %j Updated records show new values immediately

14. BEGINNER VIVA ANSWERS

Common questions you might face in viva/interview:

Q: What is Node.js?

A: Node.js is a JavaScript runtime environment that allows JavaScript code to run on servers (not just browsers). It's built on Chrome's V8 engine and is very fast for I/O operations.

Q: What is Express?

A: Express is a lightweight web framework for Node.js that simplifies creating REST APIs. It handles HTTP requests/responses, routing, middleware, etc. Makes server development much easier.

Q: What is MongoDB?

A: MongoDB is a NoSQL database that stores data in JSON-like documents instead of rows and columns. It's flexible - you can change data structure without migrations. Data is stored in collections.

Q: What is Mongoose?

A: Mongoose is a library that provides schema validation and ORM-like features for MongoDB. It helps define data structure (schemas), validate data before saving, and simplifies database operations.

Q: What is an API?

A: API (Application Programming Interface) is a way for applications to communicate. REST API uses HTTP methods (GET, POST, PUT, DELETE) on URLs to request/provide data. Our backend is a REST API.

Q: What is CRUD?

A: CRUD = Create, Read, Update, Delete. These are the four basic operations on data.

- CREATE (POST) - Add new data
- READ (GET) - Retrieve data
- UPDATE (PUT/PATCH) - Modify existing data
- DELETE (DELETE) - Remove data

Q: What is JWT (JSON Web Token)?

A: JWT is a secure token that proves a user is authenticated. After login, client receives a JWT token. This token is sent in every request to prove the user is logged in. Server verifies token using a secret key.

Q: Why do we use bcrypt?

A: bcrypt is a password hashing library. Instead of storing plain passwords, we hash them using bcrypt. This way, even if database is hacked, passwords are protected. bcrypt also adds "salt" for extra security.

Q: What is a controller?

A: Controller is a file containing functions that handle business logic. Each route points to a controller function. Controller receives request, processes it (with database), and sends response.

Q: What is a route?

A: Route maps an HTTP method + URL to a controller function. Example: POST /users/register !' registerUser() function. Routes define all available API endpoints.

Q: What is a model?

A: Model is a Mongoose schema definition. It defines what fields a collection has, their data types, and validation rules. Model represents the structure of data in MongoDB.

Q: What is middleware?

A: Middleware is code that runs between receiving request and sending response. Example: auth.js middleware checks JWT token before allowing request to reach controller.

Q: How does Postman connect to backend?

A: Postman sends HTTP requests to backend URL (http://localhost:3000/...). Backend receives request, processes it, and sends back HTTP response. Postman displays the response.

Q: How does data flow from API to MongoDB?

A: Postman !' HTTP Request !' Express Route !' Controller Function !' Mongoose Model !' MongoDB Database !' Response back to Postman

Q: What is authentication?

A: Authentication = proving you are who you claim. Usually email + password. Backend checks credentials and issues JWT token.

Q: What is authorization?

A: Authorization = what are you allowed to do after authentication. Example: Only customers can add vehicles, only garage owners can create garages.

Q: What is the difference between 401 and 403 errors?

A: 401 = Not authenticated (no token or invalid token). 403 = Authenticated but not authorized (don't have required role).

Q: What is NoSQL?

A: NoSQL = database without fixed table structure. Documents can have different fields. More flexible than SQL. MongoDB is NoSQL.

Q: What is the difference between SQL and NoSQL?

A: SQL has fixed table structure with rows/columns. NoSQL stores flexible documents. NoSQL is good for varying data. MongoDB (NoSQL) is easier for rapid development.

Q: Why use MongoDB instead of MySQL?

A: MongoDB stores data as JSON (easy to work with in JavaScript). Schema is flexible (can add fields anytime). No complex joins. Good for applications with varying data structure.

15. COMPLETE BACKEND FLOW DIAGRAM

This diagram shows how data flows through the entire backend:

[illegible]

Example: Authenticated Request (Create Vehicle)

After login, when making a request with JWT:

REQUEST (with JWT):

POST /vehicles

Authorization: Bearer <JWT_TOKEN>

FLOW:

1. Express receives request
2. Middleware (auth.js) checks Authorization header
3. Middleware verifies JWT token
4. If valid, extracts userId and role from token
5. Loads user from database, attaches to req.user
6. Route checks role: allowRoles("customer")
7. If authorized, vehicleController.addVehicle() runs
8. Controller creates Vehicle with owner = req.user._id
9. Vehicle saved to MongoDB
10. Response sent back to Postman

16. CONCLUSION & NEXT STEPS

Congratulations! You now understand your entire backend system.

What You Learned:

- ' Project structure and how files connect
- ' How Express routes work
- ' How MongoDB and Mongoose work
- ' CRUD operations
- ' Authentication and JWT tokens
- ' Authorization and roles
- ' All API endpoints
- ' How to test with Postman
- ' How to verify data in MongoDB Compass

Key Concepts to Remember:

1. Request flows: Postman !' Route !' Controller !' Model !' MongoDB !' Response
2. JWT tokens prove user is logged in
3. Each collection has multiple documents
4. Models define schema (structure) for data
5. Controllers contain business logic
6. Routes map URLs to controllers
7. Middleware runs before controllers (like auth checks)
8. Passwords are hashed with bcrypt
9. Roles control what each user can do
10. MongoDB stores data as JSON documents

Common Mistakes to Avoid:

- Forgetting to add Authorization header with JWT in Postman
- Assuming timestamps are in database by default (use { timestamps: true })
- Storing passwords as plaintext (always use bcrypt)
- Not checking user permissions (anyone could update anyone else's data)
- Forgetting to validate required fields
- Not handling errors properly
- Using wrong HTTP method (GET vs POST)

Next Steps:

1. Test the entire backend using this documentation
2. Create test data with realistic scenarios
3. Verify data in MongoDB Compass
4. Try building simple features (e.g., delete user)
5. Understand error handling
6. Learn about deployment (deploy to AWS, Heroku, etc)
7. Add more features (email verification, password reset, etc)

Resources to Learn More:

- Express.js official docs: <https://expressjs.com>
- MongoDB docs: <https://docs.mongodb.com>
- Mongoose docs: <https://mongoosejs.com>
- JWT: <https://jwt.io>
- bcrypt: <https://www.npmjs.com/package/bcrypt>